

Uplift Healthcare Business SQL Problems: 100-Question Workbook

Database: uplift_healthcare

Target Audience: Data Science and Healthcare Analytics Students

Categorization: Easy (35), Medium (35), Hard (30)

=====

=====

===== PART 1: EASY PROBLEMS (BP-001 to BP-035) Focus: Fundamental metrics, basic counts, active policy filtering, and operational snapshots.

BP-001: Active Member Count

- Business Rationale: Assessing membership volume is critical for operational forecasting and per-member-per-month (PMPM) financial underwriting.
- SQL Task: Retrieve the total count of currently active members in the 'members' table where the status is active.

BP-002: High-Premium Underwriting

- Business Rationale: Identifying premium concentration helps finance teams identify high-value policy brackets and risk tiers.
- SQL Task: Find all active records in the 'policies' table with a premium_amount strictly greater than \$1,200.

BP-003: Network Leakage Check

- Business Rationale: Out-of-network facilities drive up healthcare costs for both members and payers. Finding them helps recruiters focus provider network expansion.
- SQL Task: List the names, addresses, and types of all facilities in the 'facilities' table where the network_status is marked 'OUT-OF-NETWORK'.

BP-004: Claims Billing Concentration

- Business Rationale: Locating the highest gross-billing events allows risk managers to focus audits on high-exposure claims first.
- SQL Task: Select the claim_id, member_id, and total_billed_amount for the 10 most expensive claims in the 'claims_header' table, sorted descending.

BP-005: Operational Backlog Analysis

- Business Rationale: Prior authorizations block patient care if delayed. Backlogs must be cleared to maintain Service Level Agreements (SLAs).
- SQL Task: Retrieve all prior authorization records in 'prior_authorizations' that currently have an auth_status of 'PENDING'.

BP-006: Practitioner Pediatric Mapping

- Business Rationale: Ensuring adequate pediatric coverage is essential for family policy design and marketing.
- SQL Task: List all columns for practitioners in 'practitioners' whose specialty_id maps to pediatric codes (or filter for specific IDs).

BP-007: Out-of-Network Doctors

- Business Rationale: Patient experience drops if their doctor is out-of-network. We need a list of these doctors for outreach.
- SQL Task: Select the first_name, last_name, and license_number of all practitioners whose network_status is 'OUT-OF-NETWORK'.

BP-008: High-Dollar Auditing Snapshot

- Business Rationale: Claims above \$10k require special high-dollar committee reviews. We need to find pending audits.
- SQL Task: Find all records in the 'high_dollar_claims_review' table that are currently marked with an audit_status of 'PENDING_REVIEW'.

BP-009: Premium Revenue Cap Pricing

- Business Rationale: High-deductible plans must be analyzed to verify the ratio of monthly premiums to deductibles.
- SQL Task: List the plan_name and base_monthly_rate from 'coverage_plans' where the annual_deductible is \$5,000 or higher.

BP-010: Out-of-State Members Audit

- Business Rationale: Members living outside the home state might have limited network access, requiring specialized out-of-area PPO plans.
- SQL Task: Retrieve all member columns from 'members' where the address state is not 'TX' (assuming Texas is the domestic network).

BP-011: Zero-Value Claims Identification

- Business Rationale: A billed amount of \$0 on a claim line item often signals an intake error

or a missing itemized service code.

- SQL Task: Query 'claim_line_items' and retrieve all lines where the billed_amount is exactly 0.00.

BP-012: Top Rated Practitioners

- Business Rationale: Elevating high-performing doctors in our search directory increases patient satisfaction.
- SQL Task: Query 'provider_reviews' and list all practitioner_id values that received a perfect 5-star rating.

BP-013: Prior Authorization Volume Audit

- Business Rationale: Prior authorization quantities over a certain threshold could signal outpatient over-utilization.
- SQL Task: Select all prior authorization records where the approved_quantity is strictly greater than 5.

BP-014: Urgent Care Directory

- Business Rationale: Offering alternatives to emergency rooms lowers claim costs. We need a quick lookup of urgent cares.
- SQL Task: Select the facility_name, phone_number, and address of all facilities of type 'Urgent Care'.

BP-015: Senior Citizen Member Profile

- Business Rationale: Senior demographics drive preventative geriatric management and Medicare Advantage planning.
- SQL Task: List all columns of female members ('F') in the 'members' table who were born before January 1, 1960.

BP-016: Customer Satisfaction (CSAT) Metrics

- Business Rationale: Monitoring unresolved customer complaints helps prevent member churn and regulatory compliance audits.
- SQL Task: List the member_id, complaint_type, and current_status for all member complaints that are still 'PENDING_REVIEW'.

BP-017: Active Policy Riders Inventory

- Business Rationale: Optional riders (like dental or vision additions) are high-margin products. We must track active additions.
- SQL Task: Retrieve all policy_rider_id and policy_id values from the 'policy_riders' table that are marked 'ACTIVE'.

BP-018: Major Contractual Write-Offs

- Business Rationale: Write-offs signify negotiated discount savings. High write-off claims showcase provider contract strength.
- SQL Task: Identify all lines in 'claim_line_items' where the contractual_write_off is greater

than \$2,500.

BP-019: Primary Household Relationship Mapping

- Business Rationale: Underwriting dependents differently from main policyholders requires separating demographics.
- SQL Task: Count how many members in the 'members' table are enrolled under the relationship status 'CHILD'.

BP-020: Hospital Discharge Claims Tracking

- Business Rationale: Inpatient hospital bills drive massive financial exposure. We need to monitor hospital-level claims.
- SQL Task: Select all claims from the 'claims_header' where the facility_id matches our main inpatient facility.

BP-021: Fixed Copay Plans Analysis

- Business Rationale: Plans with low office visit copays encourage preventative visits. We must inventory these plan models.
- SQL Task: List all plans in 'coverage_plans' that have a copay_amount of exactly \$15, \$20, or \$25.

BP-022: Denial Auditing Timeline

- Business Rationale: Identifying claims denied during critical quarters helps calculate financial liability adjustments.
- SQL Task: List all claims from 'claims_header' where the claim_status is 'DENIED' and the service date was in Q1 2026.

BP-023: Premium Payment Processing Check

- Business Rationale: Cash reconciliations require auditing daily batch payment receipts.
- SQL Task: Retrieve all columns from 'premium_payments' where the payment_date is exactly '2026-06-01'.

BP-024: Long-Term Provider Contracts

- Business Rationale: Long-term contracts offer predictable costs. We need to identify contracts expiring past 2028.
- SQL Task: List all facility contracts from 'provider_contracts' where the contract_end_date is after '2028-12-31'.

BP-025: Wellness Perk Categories

- Business Rationale: Assessing wellness perk distributions tells us what incentives drive the most member interest.
- SQL Task: Find all wellness benefits in 'wellness_benefits' that offer a reward_type of 'GIFT_CARD'.

BP-026: Lower Out-of-Pocket Safeguards

- Business Rationale: Plans with low out-of-pocket maximums are attractive to chronically ill patients. We must balance our pricing pool.
- SQL Task: Find all coverage plans where the `out_of_pocket_max` is strictly below \$5,000.

BP-027: Reinsurance Stops

- Business Rationale: Large claims must trigger reinsurance recovery files before we lose cash buffer limits.
- SQL Task: Select all claims in `'reinsurance_stop_loss_claims'` where the `individual_loss_amount` exceeds \$50,000.

BP-028: Clearances Pipeline

- Business Rationale: Tracing cleared payments prevents outstanding liability discrepancies.
- SQL Task: Query `'vendor_payment_clearinghouse'` and list all records where the `cleared_timestamp` is currently NULL.

BP-029: Non-Standard Facility Scopes

- Business Rationale: Facilities operating on non-standard scopes might be performing unauthorized treatments.
- SQL Task: List all unique `specialty_id` codes associated with facilities that are marked as `'OUT-OF-NETWORK'`.

BP-030: Dependent Demographics Audit

- Business Rationale: Pediatric wellness reminders are sent based on child gender demographics.
- SQL Task: Retrieve all male dependents ('M') whose `relationship_to_holder` is `'CHILD'` from the `'members'` table.

BP-031: PPO Underwriting Exposure

- Business Rationale: PPO options have wide provider choice but higher loss ratios. We must track active policy counts.
- SQL Task: Count how many active policies in the `'policies'` table are currently linked to coverage plans of type `'PPO'`.

BP-032: Total Provider Grievances Volume

- Business Rationale: A sudden spike in grievances points to friction in our auto-adjudication systems.
- SQL Task: Count the total number of records in `'provider_grievances'` submitted in the calendar year 2026.

BP-033: Imminent Auth Expirations

- Business Rationale: Members must be notified if their approved procedure authorizations are expiring soon.
- SQL Task: Retrieve all prior authorizations where the `expiration_date` is between `'2026-06-01'` and `'2026-06-30'`.

BP-034: Coinsurance Free Plans

- Business Rationale: Plans with 0% coinsurance after deductible met are marketed as premium gold plans.
- SQL Task: List plan_name and base_monthly_rate from 'coverage_plans' where the coinsurance_percentage is exactly 0.

BP-035: Reinsurance Stop Loss Risk Analysis

- Business Rationale: Tracking stop-loss processing states is necessary to avoid paying excess claims out-of-pocket.
- SQL Task: Find all claims in 'reinsurance_stop_loss_claims' where the stop_loss_status is 'PENDING_REIMBURSEMENT'.

=====

=====

===== PART 2: MEDIUM PROBLEMS (BP-036 to BP-070) Focus: Joins, aggregated summaries, time-series, matching prior auths, and provider audits.

BP-036: Premium Collection Delinquency

- Business Rationale: If a policyholder is active but payments are lagging, we risk financial premium leakage.
- SQL Task: Join the 'policies' table with the 'premium_payments' table to calculate the total premium amount expected versus the total amount paid, grouped by policy_id.

BP-037: Member Out-of-Pocket Accumulation

- Business Rationale: Monitoring real-time cost shares helps support customer helpdesks when members inquire about their deductible progress.
- SQL Task: Join 'members' with 'member_accumulators' to select the first_name, last_name, annual_deductible, and ytd_deductible_met, filtering for members who have met at least 75% of their deductible.

BP-038: Adjudication Turnaround Time (SLA Analysis)

- Business Rationale: Claims regulations require adjudicating medical claims within a strict

threshold of days from submission to payment.

- SQL Task: Join 'claims_header' with 'facilities' to calculate the average number of days between the date_of_service and the review_date, grouped by facility_name.

BP-039: Physician Directory Mapping

- Business Rationale: Patient-facing directory apps require matching doctor credentials with their clinical board specialty descriptions.
- SQL Task: Join 'practitioners' with 'specialties' to display the practitioner first name, last name, and the full description of their medical specialty.

BP-040: Prior Authorization Match Auditing

- Business Rationale: Fraud detection requires verifying that high-cost claims actually matched an approved prior authorization.
- SQL Task: Write an INNER JOIN between 'claims_header' and 'prior_authorizations' on the prior_auth_id field to display the claim_id, total_billed_amount, procedure_code, and auth_status.

BP-041: Customer Service Ticket Resolution Performance

- Business Rationale: Grievances must be addressed quickly. We need to identify departments lagging in turnaround times.
- SQL Task: Join 'members' and 'member_complaints' to list the member names, complaint categories, and calculate the time elapsed (in days) between complaint submission and resolution.

BP-042: Coverage Validation Audits

- Business Rationale: System integrity checks ensure no members are listed as active if their corresponding policy is terminated.
- SQL Task: Write a LEFT JOIN between 'members' and 'policies' on policy_id, filtering for cases where member_status is 'ACTIVE' but policy_status is 'TERMINATED' or NULL.

BP-043: Top Disputed Facilities

- Business Rationale: High disputes indicate contract negotiation friction. We need to list facilities with high disputation amounts.
- SQL Task: Join 'facilities' and 'provider_grievances' to calculate the total disputed_amount grouped by facility_name, sorted from highest to lowest.

BP-044: Copay Under-Adjudication Check

- Business Rationale: If claims are paid without deducting the correct member copay, the insurer loses money.
- SQL Task: Join 'claim_line_items', 'claims_header', and 'coverage_plans' to identify any paid claims where the paid_amount plus copay_amount does not equal the allowed_amount.

BP-045: Outstanding Disbursements Clearing Reconciliation

- Business Rationale: Uncleared checks can lead to unclaimed property exposure if not

settled within statutory windows.

- SQL Task: Perform a LEFT JOIN between 'claim_disbursements' and 'vendor_payment_clearinghouse' on payment_reference_id to identify disbursements that have not yet cleared.

BP-046: Practitioner Specialty Billing Deviation

- Business Rationale: Outlier billing checks help identify providers who may be overutilizing procedure codes compared to their specialty peers.
- SQL Task: Join 'practitioners', 'claims_header', and 'specialties' to calculate the average total_billed_amount per claim, grouped by specialty_name and compared to individual practitioner averages.

BP-047: Household Cost Burden Evaluation

- Business Rationale: Family-tier plans need to be modeled to understand if the primary member or their child dependents use more medical services.
- SQL Task: Join 'members' with 'claims_header' to find the total sum of total_billed_amount, grouped by the relationship_to_holder category.

BP-048: Plan Popularity Trends

- Business Rationale: Knowing plan enrollment growth trends helps direct sales commissions and product designs.
- SQL Task: Join 'policies' and 'coverage_plans' to count the number of active enrollments, grouped by plan_name and ordered by popularity.

BP-049: Wellness Benefit Repeat Utilization

- Business Rationale: High wellness benefit engagement corresponds to lower overall medical risk. We must track repeat users.
- SQL Task: Join 'members' with 'member_wellness_claims' to find members who have utilized wellness benefits more than twice, grouping by member name and counting claims.

BP-050: Write-Off Success Metric

- Business Rationale: Contract strength is measured by discount depth. We need to calculate discount ratios by facility.
- SQL Task: Join 'claims_header', 'claim_line_items', and 'facilities' to calculate the average ratio of contractual_write_off to billed_amount, grouped by facility_name.

BP-051: Coordination of Benefits Leakage Avoidance

- Business Rationale: If a member has other insurance, we must ensure they are billed as primary first to avoid redundant payments.
- SQL Task: Join 'members', 'claims_header', and 'member_other_insurance' to display claims for members who have active secondary insurance policies.

BP-052: Systemic Claim Rejection Audit

- Business Rationale: Finding the root cause of automated denials helps us update provider billing guides.
- SQL Task: Join 'claim_adjudication_logs' with 'denial_reason_codes' to count the total occurrences of each rejection reason, sorted descending.

BP-053: Inpatient Readmission Window

- Business Rationale: Under CMS rules, 30-day hospital readmissions trigger penalties. We must track readmission frequencies.
- SQL Task: Self-join 'claims_header' on member_id where the first claim's date_of_service is within 30 days prior to the second claim's date_of_service, selecting the member_id and both claim dates.

BP-054: Premium Payment Aging (Aging Report)

- Business Rationale: Policy cancellations are triggered if premiums go unpaid beyond the statutory 30-day grace period.
- SQL Task: Join 'policies' with 'premium_payments' to find active policies whose last payment_date is older than 45 days.

BP-055: Specialist vs Primary Care Utilization Analysis

- Business Rationale: HMO plan rules require primary care physician referrals for specialist visits. We need to audit compliance.
- SQL Task: Join 'prior_authorizations', 'practitioners', and 'specialties' to calculate the count of auth requests, grouped by specialty_name.

BP-056: Reinsurance Recovery Yields

- Business Rationale: Stop-loss claims represent heavy losses. We need to track actual reimbursements against original losses.
- SQL Task: Join 'claims_header' with 'reinsurance_stop_loss_claims' to compare total_billed_amount against reinsurance_recovery_amount.

BP-057: Out-of-Network Financial Inflation

- Business Rationale: We need to negotiate with out-of-network providers who charge significantly higher than our standard in-network schedule.
- SQL Task: Join 'claims_header' and 'facilities' to compare average total_billed_amount grouped by network_status.

BP-058: Practitioner Complaint Frequency Correlation

- Business Rationale: Quality departments must flag practitioners who have high volumes of clinical patient complaints.
- SQL Task: Join 'practitioners', 'claims_header', and 'member_complaints' to count complaints per doctor, displaying doctor names and complaint counts.

BP-059: Federal Program Funding Ledger Checks

- Business Rationale: Government program ledgers must be audited monthly to ensure

funding balances are not in deficit.

- SQL Task: Join 'government_programs' with 'government_funding_ledger' to calculate net budget remaining per program.

BP-060: Out-of-Pocket Maximum Breaches

- Business Rationale: By law, once a member hits their out-of-pocket maximum, their coinsurance must drop to 0%.
- SQL Task: Join 'members', 'member_accumulators', and 'coverage_plans' to list members who have met their annual out_of_pocket_max limit.

BP-061: Claim Underpayment Reconciliation

- Business Rationale: Providers may file disputes if we pay less than the contractually allowed amount.
- SQL Task: Query 'claim_line_items' and calculate instances where paid_amount is strictly less than allowed_amount and patient_responsibility is 0.00.

BP-062: Unapproved Prior-Authorization Claims

- Business Rationale: Paying claims for services that had denied authorizations represents a compliance failure.
- SQL Task: Join 'claims_header' and 'prior_authorizations' to list claims associated with an auth_status of 'DENIED' or 'EXPIRED'.

BP-063: Facility Claim Intake Volume Analysis

- Business Rationale: Knowing which facilities process the highest claim counts helps allocate clinical staff for faster reviews.
- SQL Task: Join 'claims_header' and 'facilities' to display the count of claims and total billed amount, grouped by facility_name.

BP-064: Rider Premium Revenues

- Business Rationale: Evaluating rider profitability ensures premium pricing remains actuarially sound.
- SQL Task: Join 'policies', 'policy_riders', and 'riders' to calculate total additional premium revenue generated by optional riders.

BP-065: Aging Provider Disputes

- Business Rationale: Prompt payment laws require resolving provider disputes within 45 days to avoid interest penalties.
- SQL Task: Join 'facilities' and 'provider_grievances' to identify pending disputes where the submission_date is older than 45 days.

BP-066: Contract Negotiated Rate Compliance Audit

- Business Rationale: System errors can cause claims to process at historical fee schedule rates instead of new contract terms.
- SQL Task: Join 'claim_line_items', 'claims_header', and 'provider_contracts' to verify that

paid amounts do not exceed contract maximum limits.

BP-067: Wellness Value Analysis

- Business Rationale: We want to check if wellness benefits reduce overall hospitalizations.
- SQL Task: Join 'members', 'member_wellness_claims', and 'claims_header' to analyze claims counts for members with wellness accounts.

BP-068: Auditing Yield by Auditor

- Business Rationale: Evaluating auditor efficiency helps optimize our internal claim integrity teams.
- SQL Task: Join 'claims_header' and 'high_dollar_claims_review' to display the sum of recovered_savings_amount grouped by audit_status.

BP-069: Claims Density per Claim Header

- Business Rationale: High numbers of service lines per claim could indicate "unbundling" (billing parts of a single procedure separately to inflate payouts).
- SQL Task: Join 'claims_header' and 'claim_line_items' to find the average number of line items per claim_id, grouped by facility_id.

BP-070: Net Portfolio Cash Flow Analysis

- Business Rationale: Actuaries must evaluate net liquidity by comparing monthly premium income directly to claim payout disbursements.
- SQL Task: Write a query combining aggregate monthly revenues from 'premium_payments' with aggregate monthly outflows from 'claim_disbursements' (using UNION or subqueries).

=====

=====

===== **PART 3: ADVANCED PROBLEMS (BP-071 to BP-100) Focus: Window functions, recursive/self-relationships, complex CTEs, auditing anomalies, and predictive risk indicators.**

BP-071: Dynamic Deductible Transition Tracking

- Business Rationale: We need to analyze member spending behaviors after they meet their deductibles and start sharing less of the cost.
- SQL Task: Write a query using CTEs that joins 'members', 'member_accumulators', and 'claims_header' to calculate the percentage of members who transitioned from paying 100% of claims to coinsurance based on meeting their deductible, grouped by plan type.

BP-072: Cumulative Cash Flow Run-Rate

- Business Rationale: Tracking running totals of premium receipts helps executive leadership evaluate monthly budget variances.
- SQL Task: Use a window function (SUM() OVER (PARTITION BY ... ORDER BY ...)) to calculate the cumulative running total of premium revenue collected from 'premium_payments' over time, grouped by billing_period.

BP-073: Out-of-Network Cost Leakage analysis

- Business Rationale: Identifying where out-of-network leakage occurs tells network managers exactly what specialties need new contracts.
- SQL Task: Create a CTE to determine the percentage of claims cost spent on 'OUT-OF-NETWORK' facilities vs 'IN-NETWORK' facilities, grouped by specialty_name (joining specialty, practitioner, claims, and facility tables).

BP-074: Provider Claim Correction Speed SLAs

- Business Rationale: Tracking corrected claim speeds prevents provider friction and regulatory prompt-pay penalties.
- SQL Task: Calculate the exact turnaround time (in days) from the submission of a provider claim correction in 'provider_claim_corrections' to the re-adjudication date in 'claims_header', displaying a moving average grouped by provider.

BP-075: Year-End Risk exposure - Stop-Loss Proximity Analysis

- Business Rationale: We need to predict stop-loss triggers to secure our reinsurance liquidity buffers.
- SQL Task: Use window functions to calculate the cumulative total of claims paid per member over the course of the year. Display members who are within 10% of triggering their individual reinsurance threshold as stored in 'reinsurance_stop_loss_claims'.

BP-076: Cohort Cost Trends

- Business Rationale: Evaluating cost progression by enrollment year helps us price multi-year premiums accurately.
- SQL Task: Write a complex query to cohort members by the year they enrolled (from policies) and track their rolling quarterly cost average over their lifecycle using 'claim_line_items'.

BP-077: Prior Authorization Lost-Revenue Leakage

- Business Rationale: When members get authorizations but skip procedures, we lose

premium optimization.

- SQL Task: Identify approved prior authorizations from 'prior_authorizations' that expired without ever having a matching claim submitted in 'claims_header', and calculate the total estimated lost procedure revenue.

BP-078: Coordination of Benefits Deductible Audit

- Business Rationale: We must ensure secondary insurance payouts do not exceed remaining patient liabilities.
- SQL Task: Analyze claims where a member has primary secondary coverage to verify that the secondary insurer's paid amount did not exceed the deductible/copay responsibility left behind by the primary insurer's adjudication.

BP-079: Clearinghouse Transaction SLA Breach

- Business Rationale: Electronic fund transfers must settle within strict SLA guidelines. Late transfers lead to interest fees.
- SQL Task: Identify payments from 'vendor_payment_clearinghouse' where the time elapsed between sent_timestamp and cleared_timestamp exceeded 48 hours, displaying the percentage of breach events grouped by vendor.

BP-080: Overlapping Insurance Coverage Detection

- Business Rationale: Dual active policies inside our own company signal administrative errors or potential enrollment fraud.
- SQL Task: Write a query using self-joins or window functions to identify any members who have overlapping policy effective dates and termination dates in the 'policies' table.

BP-081: Recursive Household cost aggregator

- Business Rationale: Household pricing models require tracking aggregate family risk rather than individual risk.
- SQL Task: Use a self-join or recursive CTE on the 'members' table (using primary_member_id) to calculate the total aggregated medical claims cost (billed and paid) for entire families, ranking households from highest to lowest cost.

BP-082: Claim Adjudication Bottleneck Tracking

- Business Rationale: Knowing which adjudication stage takes the longest helps target system optimizations.
- SQL Task: Write a query using LEAD or LAG window functions on 'claim_adjudication_logs' to calculate the average transition duration (in hours or days) between successive status states.

BP-083: Practitioner Performance Tiering

- Business Rationale: Network tiering allows rewarding doctors with low denial rates and high satisfaction reviews with premium network designations.
- SQL Task: Classify practitioners into performance categories (Gold, Silver, Bronze) by joining 'practitioners', 'provider_reviews', and 'claims_header'. Define Gold as having >4.5

average reviews and <5% claim denial rates.

BP-084: Retroactive Contract Rate Settlement Analysis

- Business Rationale: When contract negotiations drag on, rate updates are applied retroactively. We must calculate the settlement difference.
- SQL Task: Write a query to recalculate historical claim payouts under a newly negotiated provider rate vs what was actually paid, showing the retroactive payment adjustment due to each facility.

BP-085: Retention Lift from Wellness Programs

- Business Rationale: Demonstrating that wellness participation increases retention justifies the program's marketing budget.
- SQL Task: Run a cohort analysis comparing the policy duration (months between effective and termination dates) of members who submitted wellness claims vs those who did not.

BP-086: Out-of-Pocket Cap Breach Restitution Calculation

- Business Rationale: Charging members cost shares after they have met their out-of-pocket maximum violates compliance laws. We must refund them.
- SQL Task: Write a query that identifies claims paid after a member already breached their out_of_pocket_max (using accumulators and claim dates), and calculate the total restitution amount due back to the member.

BP-087: Split-Billing Fraud Detection

- Business Rationale: Providers may split a single major surgery into minor claim lines across multiple days to bypass limits.
- SQL Task: Write a pattern matching query to identify members who had multiple claims with identical diagnosis codes submitted by different practitioners at the same facility within a 72-hour window.

BP-088: Audit savings Recovery analysis

- Business Rationale: Measuring auditor recovery rates helps us focus resources on facilities with the highest recovery yields.
- SQL Task: Join 'claims_header' with 'high_dollar_claims_review' to calculate the recovery success rate (recovered savings vs original claim amounts) for each facility, ranking facilities based on auditing leakage.

BP-089: Stop-Loss underwriting Risk Predictor

- Business Rationale: High accumulator values late in the year predict high reinsurance stop-loss claims in the next quarter.
- SQL Task: Write a query that calculates what percentage of members in each coverage plan have met 80% or more of their annual out-of-pocket maximum, flag plans that exceed a 15% density threshold.

BP-090: Year-Over-Year Procedure Inflation Analysis

- Business Rationale: Medical CPI inflation drives premium price increases. We must track inflation by procedure code.
- SQL Task: Write a query to calculate the year-over-year percentage change in the average billed_amount per procedure_code from 'claim_line_items', filtering for codes performed in both 2025 and 2026.

BP-091: Provider Network Adequacy Gaps

- Business Rationale: Regulators mandate that members must have an in-network hospital within a specific geographic range.
- SQL Task: Analyze member locations against facility addresses to identify members who do not have an in-network facility of type 'Hospital' in their home state/zip area, highlighting geographic coverage gaps.

BP-092: Pro-Rated Premium Refunds

- Business Rationale: Terminating policies mid-month requires returning pro-rated premiums to subscribers.
- SQL Task: Write a query that calculates the exact pro-rated refund amount due to members who terminated their policies, based on the termination_date, monthly premium, and days remaining in the billing period.

BP-093: Multi-System Reconciliation Audit

- Business Rationale: Discrepancies between approved claims and actual disbursements indicate potential accounting errors or payment processing failures.
- SQL Task: Write a reconciliation query to check the financial pipeline: Select all claims marked 'PAID' in 'claims_header', verify they have a matching record in 'claim_disbursements', and ensure the disbursed amount perfectly matches the allowed amount.

BP-094: Upcoding Fraud Pattern Detection

- Business Rationale: Upcoding (billing a more expensive procedure than actually performed) is a common form of healthcare billing fraud.
- SQL Task: Write a query that evaluates the distribution of evaluation and management (E&M) codes (99211 through 99215) billed by each practitioner. Flag any practitioners whose ratio of 99215 (highest complexity) to 99211 (lowest) is 3 standard deviations above the peer average.

BP-095: Customer Lifetime Value (LTV) Modeling

- Business Rationale: Knowing LTV helps marketing determine maximum customer acquisition cost limits.
- SQL Task: Create an LTV model for each policy type by calculating total premium revenue collected minus total claims paid, divided by total months enrolled, grouped by plan_name.

BP-096: Specialty Referral bypassing

- Business Rationale: Patients bypassing gatekeeper referrals drive up plan costs. We must

identify where this occurs.

- SQL Task: Write a query to flag paid specialty claims in 'claims_header' where the member's plan_type is 'HMO' but there is no matching approved prior authorization for that specialty.

BP-097: Ledger Spending moving-average

- Business Rationale: Spikes in government funding spending must be identified to avoid sudden program budget depletion.
- SQL Task: Query 'government_funding_ledger' and use a moving average window function (AVG() OVER (ORDER BY date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)) to identify monthly funding disbursements that deviate by more than 20% from the moving trend line.

BP-098: Stop-Loss payout prediction

- Business Rationale: If accumulator trends continue to escalate, we must alert reinsurance underwriters.
- SQL Task: Create a query that analyzes historical monthly growth rates of member accumulators to project which members are statistically modeled to trigger reinsurance stop-loss thresholds within the next 90 days.

BP-099: Practitioner referral-to-treatment Velocity

- Business Rationale: Patient recovery improves when referrals translate to treatment quickly. We need to measure provider diagnostic speeds.
- SQL Task: Calculate the "referral-to-treatment velocity" by measuring the number of days between a prior-authorization approval and the actual claim date_of_service, grouped by practitioner.

BP-100: Ultimate healthcare Financial balance reconciliation sheet

- Business Rationale: Ensuring complete reconciliation across the entire financial transaction life-cycle is the ultimate test of medical plan solvency.
- SQL Task: Write a comprehensive, multi-table query that aggregates and displays a master balance sheet for the entire database: Total Premiums Billed, Total Premiums Collected, Total Claims Billed, Total Allowed, Total Paid, Total Disbursed, and Total Cleared, grouped by plan_name.

```
=====
=====
END OF WORKBOOK
=====
=====
```